

COVER_TIME

Overview

cover_time collects a range timing coverage information while monitoring the time difference between two given signals. Users may define any number of time ranges (time is measured in clock cycles). Each time range has a unique name, the minimum and maximum limits, and a coverage goal, defining the minimum number of hits within this range. The cover_time verification component is able to generate coverage reports, dynamically supply the accumulated coverage number to the reactive testbench components and report about the full coverage accumulation asserting the “covered” event.

Syntax

```
cover_time #(cover_num, ssize) InstName (clk, rst_n, start_event, end_event);
```

Parameters		
Cover_num		Maximum number of coverage ranges to be defined. Default: 0
Ssize		String size for the name of cover item. Default: 20
Interface		
clk		Sampling clock for data
rst_n		Active-low reset
start_event		The pulse of start_event defines the start of time interval to be covered
end_event		The pulse of end_event defines the end of time interval to be covered
Access Tasks		
Name	Arguments	Description
reset	No	Resets coverage counters and maximum & minimum limits
add	cover_name min_range max_range cover_goal	- name of the coverage Item (with length <= ssize) - minimum limit of timing range - maximum limit of timing range - coverage: minimum number of hits into this interval Example: <InstName>.add("Small", 3 5, 8); – The distance between the start and end signals has to be 8 times within the range [3:5] clocks.
change	cover_name min_range max_range cover_goal	- name of the coverage Item (with length <= ssize) - minimum limit of timing range - maximum limit of timing range - coverage: minimum number of hits into this interval Note: The name of the coverage range has to be defined at this point Example: <InstName>.change("Small", 4 7, 9); – Distance between the start and end signals now is to 9 times within the range [4:7] clocks.
report	No	Generates coverage report in tabular format. See below an example of coverage report.
Access Events		
covered		“Fires” when all coverage range goals are succeeded. Can be monitored externally to define the length of the test case execution based on accumulated coverage information.

Access Variables	
total_cov	Provided a constantly updated value of the total coverage (in %). After a “covered” event, it has to be equal to 100 %.
Control Variables	
Display_status	Set it to “1” to get more informational messages. Default: 0

Verilog example

The example below shows how coverage information controls the simulation run. By monitoring a “covered” event, the test case is running until a defined coverage is accumulated.

```

module test;

reg clk; initial clk = 0;
reg [2:0] data;

parameter INIT = 0;
parameter S1 = 1;
parameter S2 = 2;
parameter S3 = 3;
parameter S4 = 4;
parameter S5 = 5;
parameter S6 = 6;
parameter S7 = 7;

always #10 clk <= ~clk;

always @(posedge clk)
    data <= ($random)%7;

//-----
cover_time #(3) cov (clk, rst_n, (data == S2), (data == S5));

initial begin
    cov.display_status = 0;
    //      name          min  max goal
    //      -----
    cov.add("Low Bound", 1, 4, 10);
    cov.add("Middle", 5, 9, 10);
    cov.add("High Bound", 10, 20, 10);
end
//-----

initial forever begin
    #1000 cov.report;
end

// Run till all functional coverage is collected; then exit
always @(cov.covered) begin
    $display("[%0t] TC INFO: Stimulus coverage reached, test may exit", $time);
    cov.report;
    $finish;
end

//If functional coverage is not collected, wait for timeout
initial begin
    #40000 $finish;
end

endmodule

```